

Rapport Systèmes d'Exploitation Centralisés

Prune MAMALET

Mai 2025

TP1

Etape 1 : Testez le programme

Etape 2 : Lancement de la commande

- **Objectif** : pouvoir lancer une commande saisie dans un processus fils
- **Réalisation** : Ajout de la commande `execvp(cmd[0],cmd)` dans le cas où le pid vaut -1, ie lorsqu'on est dans le processus fils.

Etape 3 : Enchaînement séquentiel des commandes

- **Objectif** : que le père attende la fin de l'exécution de son processus fils
- **Réalisation** : Ajout de d'un `wait(&status)` dans le cas par défaut sur le pid (lorsqu'on est dans le processus père).
- Lorsqu'on lance un processus fils avec un commande comme `sleep(10)`, il attend la fin de son exécution avant de laisser la possibilité de taper une nouvelle commande.

Etape 4 : Lancement de commandes en tâche de fond

- **Objectif** : avoir la possibilité de lancer une tâche de fond (en ajoutant un `&` en fin de ligne)
- **Réalisation** : Ajout dans le cas du père, un test sur la présence ou non d'une commande `--> background`. Si cette commande est présente on attend la fin de l'exécution du fils.
- L'affichage du caractère `>` après 10 secondes signifiait qu'il n'attendait une nouvelle commande que lorsque le sleep était terminé.
- **Question**

```
pmt1554@heidli:~$ ps -fjn
  PPID   PID   PGID   SID TTY      TPGID STAT  UID    TIME COMMAND
1343189 1343203 1343203 1343203 pts/1    1346582 Ss     58018  0:00 bash
1343203 1346582 1346582 1343203 pts/1    1346582 R+     58018  0:00 \_ ps -fjn
1327605 1331045 1331045 1331045 pts/0    1346524 Ss     58018  0:00 /usr/bin/bash
1331045 1346524 1346524 1331045 pts/0    1346524 S+     58018  0:00 \_ ./minishe
1346524 1346558 1346524 1331045 pts/0    1346524 S+     58018  0:00 \_ sleep

pmt1554@heidli:~$ ps -fjn
  PPID   PID   PGID   SID TTY      TPGID STAT  UID    TIME COMMAND
1343189 1343203 1343203 1343203 pts/1    1346626 Ss     58018  0:00 bash
1343203 1346626 1346626 1343203 pts/1    1346626 R+     58018  0:00 \_ ps -fjn
1327605 1331045 1331045 1331045 pts/0    1346524 Ss     58018  0:00 /usr/bin/bash
1331045 1346524 1346524 1331045 pts/0    1346524 S+     58018  0:00 \_ ./minishe
1346524 1346558 1346524 1331045 pts/0    1346524 S+     58018  0:00 \_ sleep
1346524 1346601 1346524 1331045 pts/0    1346524 S+     58018  0:00 \_ sleep

pmt1554@heidli:~$ ps -fjn
  PPID   PID   PGID   SID TTY      TPGID STAT  UID    TIME COMMAND
1343189 1343203 1343203 1343203 pts/1    1346722 Ss     58018  0:00 bash
1343203 1346722 1346722 1343203 pts/1    1346722 R+     58018  0:00 \_ ps -fjn
1327605 1331045 1331045 1331045 pts/0    1346524 Ss     58018  0:00 /usr/bin/bash
1331045 1346524 1346524 1331045 pts/0    1346524 S+     58018  0:00 \_ ./minishe
1346524 1346558 1346524 1331045 pts/0    1346524 S+     58018  0:00 \_ sleep

pmt1554@heidli:~$ ps -fjn
  PPID   PID   PGID   SID TTY      TPGID STAT  UID    TIME COMMAND
1343189 1343203 1343203 1343203 pts/1    1346746 Ss     58018  0:00 bash
1343203 1346746 1346746 1343203 pts/1    1346746 R+     58018  0:00 \_ ps -fjn
1327605 1331045 1331045 1331045 pts/0    1346524 Ss     58018  0:00 /usr/bin/bash
1331045 1346524 1346524 1331045 pts/0    1346524 S+     58018  0:00 \_ ./minishe

pmt1554@heidli:~$
```

On observe que tant que le processus n'a pas complètement fini d'exécuter les commandes en arrière plan, il apparaît dans la liste des processus. Il ne disparaît que quand il a fini de s'exécuter (même si on a lancé un nouveau processus en premier plan par dessus).

TP2

Etape 5 : Traitement du signal SIGCHLD

Etape 6 : Utilisation de SIGCHLD pour traiter la terminaison des processus fils

- **Objectif** : pouvoir récupérer le status du processus qui vient de se terminer à l'aide de SIGCHLD
- **Réalisation** :
 - Ajout d'une fonction traitement, qui prend en entrée un signal reçu et qui affiche de le pid du fils lorsque celui ci est terminé
 - Cette fonction est enregistrée dans un struct sigaction
 - Elle est associée à la réception du signal SIGCHLD grâce à la fonction sigaction.

Etape 7 : Attendre le signal : pause

Remplacement du waitpid par un pause.

Test avec un sleep 10 & puis un sleep 50, on constate qu'il réalise la même chose que précédemment.

Etape 8 : Suspension et reprise d'un processus en arrière-plan

Etape 9 : Affichage d'un message indiquant le signal reçu

- **Objectif** : Afficher un message lorsque le processus fils est terminé, suspendu ou repris.
- **Réalisation** : Dans la fonction de traitement, on ajoute l'affichage d'un message en fonction de l'état du status (WIFEXITED, WIFSTOPPED ou WIFSIGNLED) en donnant le pid du fils dont le status a changé.

TP3

Etape 10 : Test de la frappe au clavier de ctrl-C et ctrl-Z

Lorsque l'on fait un ctrl-C, cela interrompt tout les processus y compris le minishell. Lorsque l'on fait un ctrl-Z, cela met en pause tout les processus (minishell compris).

Etape 11 : Gestion de la frappe au clavier de ctrl-C et ctrl-Z

- **Etape 11.1** : On associe la fonction traitement (via le sigaction), pour les signaux SIGINT et SIGTSTP.
- **Etape 11.2** : On crée un sigaction en utilisant comme handler le SIG_IGN qui permet d'ignorer le signal. Et on l'associe aux signaux SIGTSTP et SIGINT.
- **Etape 11.3** : On initialise un sigset_t (un ensemble de signaux) dans lequel on ajoute les signaux SIGTSTP et SIGINT. On utilise sigprocmask qui vient masquer les signaux contenu dans notre set.

Etape 12 : Détacher les processus fils en arrière plan

A la suite du sigprocmask on vient ajout un setpgid qui associe un nouveau groupe au processus appelant. Il nous permet que seul les processus en avant plan reçoivent le signaux SIGINT et SIGTSTP.

On a donc un ctrl-C qui vient couper tout les processus en avant plan dans le minishell. Et un ctrl-Z qui vient les mettre en pause.

TP4

Etape 13 : Redirections

- **Objectif** : Pouvoir associer l'entrée standard ou la sortie standard d'une commande à un fichier
- **Réalisation** :

```

/*redirection des entrées sorties*/
if (commande->out != NULL) {
    char* f = commande->out;
    int fdest = open(f,O_WRONLY | O_CREAT | O_TRUNC,0644);
    dup2(fdest,1);
}
if (commande->in != NULL) {
    char* f = commande->in;
    int fdssource = open(f,O_RDONLY);
    dup2(fdssource,0);
}

```

Figure 1: Caption

- Il est bien possible de de rediriger la sortie d'une commande dans un fichier donné, par exemple : avec `ls > fichier1` qui créer le fichier1 et liste le contenu du repertoire courant dedans.

Etape 14 : Déplacement dans l'arborescence

- **Objectif** : Pouvoir changer de repertoire courant vers un repertoire donné en argument
- **Réalisation** : Ajout d'une fonction appelée lorsque la commande `cd` est tapée dans le terminal.

```

void changerRep(char* chemin) {
    if (chemin == NULL) {
        chemin = getenv("HOME");
    }
    chdir(chemin);
}

```

```

if (strcmp(cmd[0], "cd")==0) {
    changerRep(cmd[1]);
}

```

- Il est bien possible de changer de repertoire courant lorsqu'on utilise la commande `cd`

Etape 15 : Afficher le contenu d'un repertoire

- **Objectif** : Pouvoir afficher le contenu du repertoire actuel ou donné en argument à l'aide de la commande `dir`
- **Réalisation** : Ajout d'une fonction appelée lorsque la commande `dir` est tapée dans le terminal

```

void afficherRep(char* chemin){
    if (chemin==NULL) {
        chemin = getenv("PWD");
    }
    DIR *dir;
    if ((dir = opendir(chemin))== NULL) {
        perror("pb ouverture repertoire");
    }
    struct dirent *entree;
    while ((entree =readdir(dir))!= NULL){
        printf("%s\n",entree->d_name);
    }
}

```

```

> dir
.
..
f1
.vscodc
a.out
readcmd.o
minishell.o
Makefile
erreur
copier.c
tube1
readcmd.h
test_readcmd.c
minishell.c
minishell
readcmd.c
tubes
> █

```

Etape 16 : Tubes simples

- **Objectif** : Permettre de composer les commandes en les reliant par un tube
- **Réalisation** : [Je n'ai pas réussi à aboutir à la fin de cette étape par manque de temps]
 - définition d'une variable qui détermine si plusieurs commandes sont présentes en ligne de commande
 - si cette variable est vraie, on crée un tube à l'aide de pipe
 - dans le fils on vient fermer le côté du tube qui n'est pas nécessaire et l'on vient rediriger la sortie standard vers le tube à l'aide de dup2
 - on vient fermer le tube